This can lead to almost never-ending loading and unloading of pages. Algorithms have been developed to guard against this never-ending loading and unloading but the problem can still occur.

KEY TERM

Disk thrashing: when paging is being used and a repetitive state has been reached where loading one page causes a need for another page to be loaded almost immediately but the loading of this new page causes the same immediate need

Extension Question 20.01

Find out information about the virtual memory capacity and page size of a computer system that you use. In what circumstances might these need to be changed?

20.05 Operating system facilities provided for the user

The user interface may be made available as a command line, a graphical display or a voice recognition system. But the function of a user interface is always to allow the user to interact with running programs. When a program involves use of a device, the operating system provides the device driver: the user just expects the device to work. (You might, however, wish to argue that printers do not always quite fit this description.)

The operating system will provide a file system for a user to store data and programs. The user has to choose filenames and organise a directory (folder) structure but the user does not have to organise the physical data storage on a disk. If the user is a programmer, the operating system supports the provision of a programming environment. The operating system allows a program to be created and run without the programmer being familiar with how the processor functions.

When a program is running it can be considered to be a type of user. The operating system provides a set of system calls that provide an interface to the services it offers. For instance, if a program specifies that it needs to read data from a file, the request for the file is converted into a system call that causes the operating system to take charge, find the file and make it available to the program. An extension of this concept is when an operating system provides an application programming interface (API). Each API call fulfils a specific function such as creating a screen icon. The API might use one or more system calls. The API concept aims to provide portability for a program, where a program can run on different operating systems with minimal changes.

20.06 Translation software

Chapter 8 (Section 8.05) provided an overview of how a compiler or an interpreter is used. We will now will consider some details of how a compiler works with a brief reference to the workings of an interpreter. Compiler or interpreter writing is a specialised task carried out by professionals who each will have their own particular methods. So, we will just look at some general principles and illustrations of some of the techniques that are likely to be used.

A compiler can be described as having a 'front end' and a 'back end'. The front-end program performs analysis of the source code and unless errors are found produces an intermediate code that expresses completely the semantics (the meaning) of the source code. The back-end program then takes this intermediate code as input and performs synthesis of object code. This analysis–synthesis model is represented in Figure 20.07.

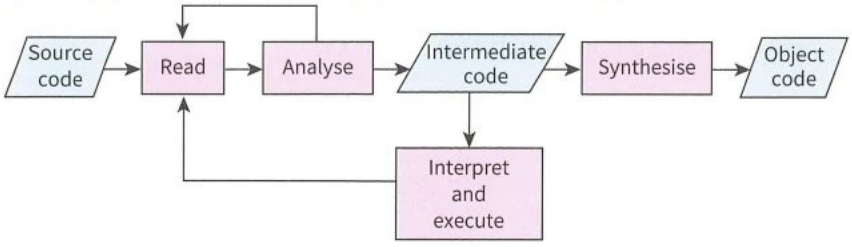


Figure 20.07 Analysis–synthesis model for a compiler

For simplicity, Figure 20.07 assumes no error in the source code. There is a repetitive process in which the source code is read line-by-line. For each line, the compiler creates matching intermediate code. Figure 20.07 also shows how an interpreter program would have the same analysis front-end: In this case, however, once a line of source code has been found to be error free and therefore converted to intermediate code, the line of source code is executed.

Front-end analysis stages

The four stages of front-end analysis, shown in Figure 20.08, are:

- lexical analysis
- syntax analysis
- semantic analysis
- intermediate code generation.

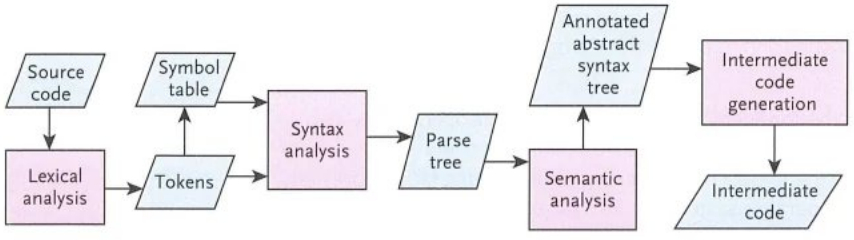


Figure 20.08 Front-end analysis

The course code that is the input data for a compiler or interpreter consists of a sequence of characters. Each meaningful individual character or collection of characters is referred to as a lexeme. A lexeme may be an identifier used by the programmer or may be a keyword, operator or symbol that is defined by the programming language. One approach to lexical analysis is first to remove all white space and all comments then to take each line of source code and identify each lexeme. This is a pattern-matching exercise. It requires the analyser to have knowledge of the components that can be found in a program written in the particular programming language.

For example, the declaration statement:

```
Var Count : integer;
```

would be recognised as containing five lexemes:

```
Var Count : integer ;
```

The assignment statement:

```
PercentMark[Count] := Score * 10
```

would be recognised as containing eight lexemes:

```
PercentMark [ Count ] := Score * 10
```

The lexical analyser must now categorise each lexeme in order to tokenise the line of code. For instance, in the first example, `Var` and `integer` must be recognised as keywords; `Count` recognised as an identifier and `:` and `;` must be recognised as distinct lexemes.

For each identifier recognized there must be an entry made in the **symbol table** (which could have been called the identifier table). The symbol table contains identifier attributes such as the data type, where it is declared and where it is assigned a value. The symbol table is an important data structure for a compiler. Although Figure 20.08 shows it only being used by the syntax analysis program, it is also used by later stages of compilation. Furthermore, most compilers are multi-pass allowing the contents of the symbol table to be frequently upgraded.

KEY TERM

Symbol table: a data structure in which each record contains the name and attributes of an identifier

Question 20.01

A symbol table is used by an assembler and by a compiler or interpreter. What are the differences between these?

Another table is used to identify the relevant token for each lexeme that is not an identifier so that the lexeme in the line of code can be replaced by a token. In some cases each identifier is also replaced by a token. Whatever scheme is applied the output from lexical analysis is a tokenised version of the source code. A wide variety of formats for the representation of tokens are mentioned in the literature.

Question 20.02

Can you think of any reason why a compiler might wish to tokenise white space rather than removing it all?

Syntax analysis, which is also known as parsing, involves analysis of the program constructs. The results of the analysis are recorded as a syntax or parse tree. Figure 20.09 shows the parse tree for the following assignment statement:

```
y := 2 * x + 4
```

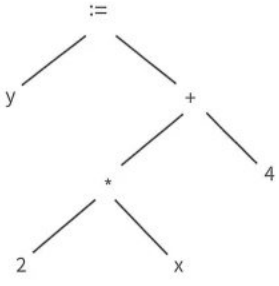


Figure 20.09 Parse tree for an assignment statement

Note that the hierarchical structure of the tree, if correctly interpreted, ensures that the multiplication of 2 by `x` is carried out before the addition of 4.

Semantic analysis is about establishing the full meaning of the code. An annotated abstract syntax tree is constructed to record this information. For the identifiers in this tree an associated set of attributes is established including, for example, the data type. These attributes are also recorded in the symbol table.

An often-used intermediate code created by the last stage of front-end analysis is a three-address code. As an example the following assignment statement has five identifiers requiring five addresses:

```
y := a + (b * c - d) / e
```

The assignment statement could be converted into the following four statements, each requiring at most three addresses:

```
temp := b * c
temp := temp - d
temp := temp / e
y := a + temp
```

Representation of the grammar of a language

For each programming language, there is a defined grammar. This grammar must be understood by a programmer and also by a compiler writer.

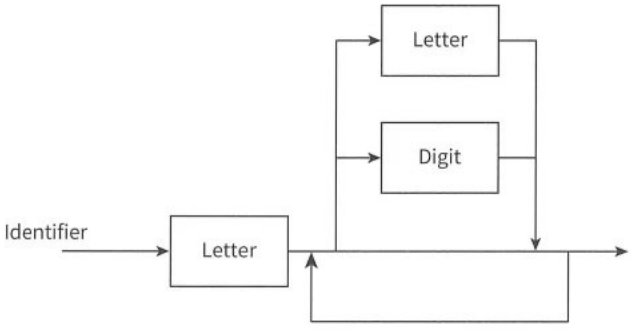


Figure 20.10 Syntax diagram defining an identifier

One method of presenting the grammar rules is a syntax diagram. Figure 20.10 represents the grammar rule that an identifier must start with a letter which can be followed by any combination of none or more letters or digits. The convention used here is that options are drawn above the main flow line and repetitions are drawn below it.

An alternative approach is to use Backus–Naur Form (BNF). A possible format for a BNF definition of an identifier is:

<Identifier> ::= <Letter>|<Identifier><Letter>|<Identifier><Digit>

<Digit> ::= 0|1|2|3|4|5|6|7|8|9

<Letter> ::= <UpperCaseLetter>|<LowerCaseLetter>

<UpperCaseLetter> ::= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z

<LowerCaseLetter> ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z

The use of | is to separate individual options. The ::= characters can be read as 'is defined as'. Note the recursive definition of <Identifier> in this particular version of BNF. Without the use of recursion the definition would need to be more complicated to include all possible combinations following the initial <Letter>.

A syntax diagram is only used in the context of a language. It has limited use because it cannot be incorporated into a compiler program as an algorithm. By contrast, BNF is a general approach which can be used to describe any assembly of data. Furthermore, it can be used as the basis for an algorithm.

Back-end synthesis stages

If the front-end analysis has established that there are syntax errors, the only back-end process is the presentation of a list of these errors. For each error, there will be an explanation and the location within the program source code.

In the absence of errors, the main back-end stage is machine code generation from the intermediate code. This may involve optimisation of the code. The aim of optimisation is to create an efficient program. One type of optimisation focuses on features that were inherent in the original source code and have been propagated into the intermediate code. As a simple example, consider these successive assignment statements:

```
x := (a + b) * (a - b)
y := (a + 2 * b) * (a - b)
```

The most efficient code would be:

```
temp := (a - b)
x := (a + b) * temp
y := x + temp * b
```


Question 20.03

Check the maths for the efficient code defined above.

Another example is when a statement inside a loop, which is therefore executed for each repetition of the loop, does the same thing each time. Optimisation would place the statement immediately before the loop.

The other type of optimisation is instigated when the machine code has been created. This type of optimisation may involve efficient use of registers or of memory.

Evaluation of expressions

An assignment statement often has an algebraic expression defining a new value for an identifier. The expression can be evaluated by first converting the infix representation in the code to Reverse Polish Notation (RPN). RPN is a postfix representation which never requires brackets and has no rules of precedence.

WORKED EXAMPLE 20.01

Manually converting an expression between RPN and infix

Converting an expression to RPN

We consider a very simple expression:

```
a + b * c
```

(Continued)

The conversion to RPN has to take into account operator precedence so the first step is to convert `b * c` to get the intermediate form:

```
a + b c *
```

We then convert the two terms to give the final RPN form:

```
a b c * +
```

If the original expression had been `(a + b) * c` (where the brackets were essential) then the conversion to RPN would have given:

```
a b + c *
```

Converting an expression from RPN

Consider this more complicated example of an RPN expression:

```
x 2 * y 3 * + 6 /
```

The RPN is scanned until two identifiers are followed by an operator. This combination is converted to give an intermediate form (brackets are used for clarification):

```
(x * 2) y 3 * + 6 /
```

This process is repeated to give the following successive versions:

```
(x * 2)(y * 3) + 6 /

(x * 2) + (y * 3) 6 /

((x * 2) + (y * 3)) / 6
```

Because of the precedence rules, some of the brackets are unnecessary; the final version could be written as:

```
(x * 2 + y * 3) / 6
```


WORKED EXAMPLE 20.02

Using a syntax tree to convert an expression to RPN

In the syntax analysis stage, an expression is represented as a syntax tree. The expression `a + b * c` would be presented as shown in Figure 20.11.

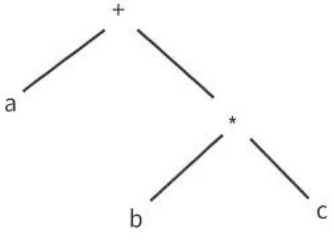


Figure 20.11 Syntax tree for an infix expression

To create this tree, the lowest precedence operator (+) is positioned at the root. If there are several with the same precedence, the first one is used. The RPN form of the expression can now be extracted by a post-order traversal. This starts at the lowest leaf to the left of the root and then uses left–right–root ordering which ensures, in this case, that the RPN representation is:

```
a b c * +
```

WORKED EXAMPLE 20.03

Using a stack with an RPN expression

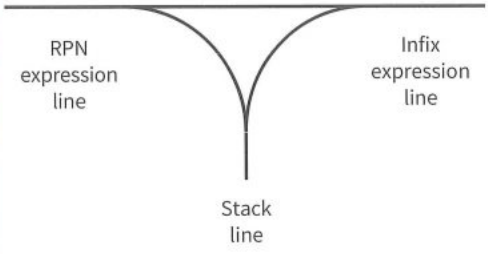


Figure 20.12 Shunting-yard algorithm

To convert an infix expression to RPN using a stack, the shunting-yard algorithm is used (Figure 20.12).

Converting an expression to RPN

The rules of the algorithm are to consider the string of tokens representing the infix expression. These represent the railroad wagons that are to be shunted from the infix line to the RPN line. The tokens are examined one by one. For each one, the rules are as follows.

- If it is an identifier, it passes straight through to the RPN expression line.
- If it is an operator, there are two options.
  - If the stack line is empty or contains a lower precedence operator, the operator is diverted into the stack line.
  - If the stack line contains an equal or higher preference operator, then that operator is popped from the stack into the RPN expression line and the new operator takes its place on the stack line.
- When all tokens have left the infix line, the operators remaining on the stack line are popped one by one from the stack line onto the RPN expression line.

Consider the infix expression `a + b * c`. Table 20.02 traces the conversion process. The first operator to enter the stack line is the + so when the higher precedence * comes later it too enters the stack line. At the end the * is popped followed by the +.

Infix line	Stack line	RPN line
`a + b * c`		
`+ b * c`		`a`
`b * c`	`+`	`a`
`* c`	`+`	`a b`
`c`	`+ *`	`a b`
	`+ *`	`a b c`
	`+`	`a b c *`
		`a b c * +`

Table 20.02 Trace of the conversion process

Had the infix expression been `a * b + c` then * would have been first to enter the stack line but it would have been popped from the stack before + could enter.

Evaluating an RPN expression

A stack can be used to evaluate an RPN expression. Let's consider the execution of the following RPN expression when x has the value 3 and y has the value 4:

```
x 2 * y 3 * + 6 /
```

The rules followed here are that the values are added to the stack in turn. The process is interrupted if the next item in the RPN expression is an operator. This causes the top two items to be popped from the stack. Then the operator is used to create a new value from these two and the new value is added to the stack. The process then continues. Figure 20.13 shows the successive contents of the stack with an indication of when an operator has been used. The intermediate states of the stack when two values have been popped are not shown.

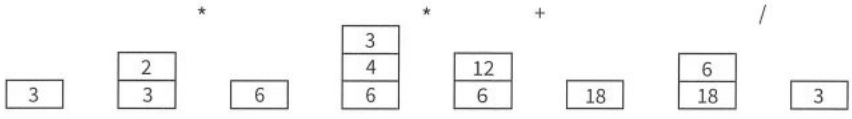


Figure 20.13 Evaluating a Reverse Polish expression using a stack

TASK 20.01

Practise your understanding of RPN.

1 Convert the following infix expressions into RPN using the methods described in Worked Examples 20.01, 20.02 and 20.03:

```
(x - y) / 4
3 * (2 + x / 7)
```

2 Convert the following RPN expressions into the corresponding infix expressions:

```
4 a b + c + d + e + *
y 2 ^ z 3 ^ + 6/
```

Note that the caret (^) symbol represents 'to the power of'.

3 Using simple values for each variable in part 2, use the infix version to evaluate the expression. Then use the stack method to evaluate the RPN expression and check that you get the same result.

The use of RPN would be of little value if the simple processor with a limited instruction set discussed in Chapter 6 was being used. Modern processors will have instructions in the instruction set that handle stack operations, so a compiler can convert expressions into RPN knowing that conversion to machine code can utilise these and allow stack processing in program execution.

Reflection Point:

Have you learned that two data structures illustrated here, the binary tree and the stack, have many different uses in computer science?